ChatGPT

1. Présentation générale du projet et objectifs

Le projet consiste à concevoir et réaliser une **plateforme de paiement numérique** adaptée au contexte africain (Bénin), permettant aux utilisateurs de payer, transférer et gérer de l'argent via une application web (Next.js) et une application mobile (Flutter), appuyées par un backend NestJS (API principale) et un back-office administratif (React Admin ou équivalent). L'objectif principal est de fournir un service de paiement moderne, sécurisé et inclusif, répondant aux spécificités locales (intégration Mobile Money, multi-devise, régulations BCEAO, KYC). La plateforme vise à :
- **Simplifier les transactions** pour les utilisateurs (citadins, ruraux) en évitant les files d'attente bancaires.
- **Augmenter l'inclusion financière** en intégrant les solutions de Mobile Money locales (Orange Money, MoMo, etc.), très répandues en Afrique de l'Ouest [1].
- **Offrir des paiements marchands flexibles**, via QR code et lien de paiement, pour faciliter le commerce électronique et de proximité.
- **Garantir la conformité réglementaire** (normes BCEAO, PCI DSS pour les cartes, KYC/AML).
- **Assurer la sécurité** (authentification forte, protection des données, chiffrement) et la robustesse (haute disponibilité, scalabilité) du système.

Les enjeux spécifiques incluent la gestion de devises multiples (au moins XOF et NGN), le paramétrage des frais de transaction selon les types d'utilisateurs, et un « cercle de confiance » simplifié (groupes d'utilisateurs autorisant des transactions entre eux). Le livrable visé est un MVP opérationnel couvrant ces fonctionnalités clés, accompagné de documentation technique et fonctionnelle complète.

2. Utilisateurs cibles et acteurs du système

La plateforme s'adresse principalement aux profils suivants :
- **Utilisateurs finaux (clients particuliers)** : personnes physiques ou morales utilisant le service pour payer des factures, transférer de l'argent à d'autres utilisateurs ou réaliser des paiements marchands. Ils s'inscrivent, créent un compte (avec vérification KYC éventuelle) et disposent d'un **wallet numérique** pour gérer leur solde.
- **Commerçants et professionnels** : entités commerciales (petits commerces, e-commerçants, prestataires de services) qui acceptent les paiements via QR code ou lien. Chaque marchand dispose d'un sous-compte ou profil marchand, avec des droits particuliers (tarifs de transaction, statuts).
- **Administrateurs / Gestionnaires (Back-office)** : personnel interne ou prestataires IT chargés de superviser la plateforme. Rôles typiques : *Admin technique* (gestion de l'infrastructure, monitoring), *Admin financier* (paramétrage des frais, supervision des transactions), *Agent KYC* (vérification manuelle des documents d'identité), support client.
- **Opérateurs de Mobile Money / Banques partenaires** : bien qu'externes à la plateforme, ils sont des partenaires d'intégration (via API). Par exemple, les services d'Orange Money, MTN Mobile Money, ou des passerelles (PayDunya, CinetPay) pour dépôt/retrait, et potentiellement des réseaux de cartes bancaires (Visa, Mastercard) via un prestataire agréé.
- **Autorités de régulation** (BCEAO, *Commission UEMOA*, ARCEP, etc.) : intervenant indirectement par leurs normes.

Ces acteurs interagissent via les différents modules du système. Les utilisateurs finaux accèdent au service par l'**application Web Next.js** ou l'**application mobile Flutter**. Les commerçants peuvent

utiliser une version dédiée (via web ou mobile) pour créer des QR codes et suivre leurs paiements. Le back-office (React Admin ou app web) est réservé aux administrateurs pour gérer les comptes, visualiser les statistiques globales, traiter les demandes KYC, etc. Les schémas de flux typiques sont : utilisateur ↔ API ↔ base de données / services externes (SMS, Mobile Money), et administrateur ↔ back-office ↔ API.

3. Architecture logique du système

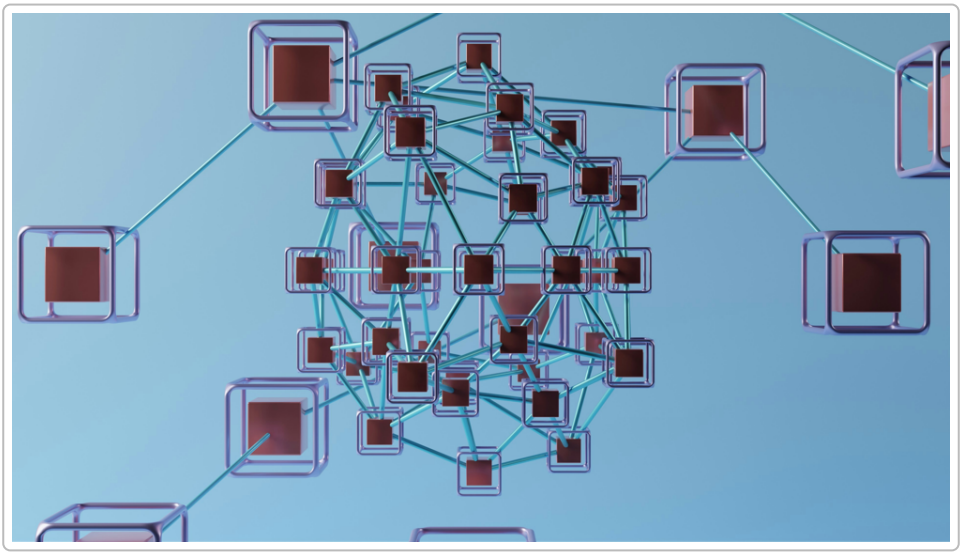


Figure 1. Architecture logicielle globale (conceptuelle) de la plateforme de paiement.

L'architecture cible est une architecture web moderne *client-serveur* composée de trois couches principales : - **Frontends** : - *Application Web* (Next.js) pour les utilisateurs sur navigateur, offrant interface de paiement, tableau de bord utilisateur, génération de QR code, etc. - *Application Mobile* (Flutter) pour iOS/Android, proposant les mêmes fonctionnalités côté utilisateur. Flutter permet un développement cross-platform rapide et un UX mobile natif. - *Back-office Administratif* (React Admin ou application web dédiée) réservé aux administrateurs pour la gestion globale (statistiques, KYC, support).
- **Backend (API principale)** : - Un service NestJS (Node.js) exposant des API REST (ou GraphQL) sécurisées, faisant le lien entre les frontends et les composants métier. Ce backend implémente toute la logique (authentification, gestion des comptes, wallet, transactions, workflows de groupe, calcul de frais, traitement des notifications, etc.). - Il communique avec la base de données centrale (p. ex. PostgreSQL ou MongoDB) pour stocker les utilisateurs, soldes de wallet, transactions, configurations, logs, etc.
- Il appelle des services externes : API Mobile Money (intégrateur ou directement vers Orange Money, MoMo…), passerelle SMS (Twilio ou local pour OTP et notifications SMS), service d'envoi d'e-mails (SMTP/SendGrid), services de push (Firebase Cloud Messaging).
- Il expose aussi des webhooks ou API pour recevoir les notifications asynchrones (statuts de paiement ou de retrait Mobile Money, retours de passerelles de paiement).
- **Infrastructure** : - Hébergement cloud (AWS, Azure, Google Cloud ou hybride), avec orchestration (Docker/Kubernetes) pour la haute disponibilité. Un pare-feu applicatif (WAF) protège les API des attaques courantes. - Cache (Redis) pour stocker temporairement OTP, sessions ou données fréquemment lues.
- Système de surveillance et logs (Prometheus, Grafana, ELK) pour la maintenance et le support.

Dans cet ensemble, le frontend Next.js et l'app Flutter consomment l'API NestJS via HTTPS. Le back-office s'authentifie également auprès de cette API (comptes admin dédiés). L'API NestJS orchestre le traitement de bout en bout, y compris la communication avec les prestataires externes (par ex. appel d'API Orange Money pour débiter/créditer un portefeuille externe) ou avec les passerelles de paiement de cartes. L'isolation des rôles est assurée par des politiques d'accès (Rôles et Permissions définis dans l'API).

Cette architecture modulaire et distribuée permet de découpler le développement (front, mobile, API, back-office), d'assurer la scalabilité horizontale (instances du service Node.js mises en cluster), et de se conformer aux exigences de résilience et de sécurité du secteur fintech.

4. Découpage modulaire

Le périmètre fonctionnel du MVP inclut les modules suivants, détaillés ci-dessous :

1. **Authentification sécurisée (OTP/MFA)** :
2. Inscription et connexion des utilisateurs avec **double facteur**. Après saisie du mot de passe (ou email), un code OTP (One-Time Password) est envoyé par SMS (ou email) pour valider l'accès [2]. Les OTP sont courts et valides quelques minutes, selon les bonnes pratiques (TOTP ou HOTP) pour réduire les risques d'interception [3].
3. Gestion du renouvellement des OTP (re-sending), verrouillage de compte après X tentatives, et fonctionnalités de *password reset* sécurisé via OTP.
4. Support éventuel de méthodes plus avancées (authenticator apps, FIDO2/WebAuthn) dans les évolutions ultérieures.

5. **Raison** : garantir un niveau de sécurité élevé pour l'accès aux comptes financiers. On note toutefois que l'OTP par SMS reste vulnérable aux attaques par échange de SIM ou phishing [4] ; d'où la recommandation de surveiller l'authentification (limitation de tentatives, détection de fraude) même si le code OTP est requis.

6. **Création et gestion de comptes utilisateurs** :

7. Inscription simplifiée par numéro de téléphone ou email. Collecte des données de base (nom, prénom, numéro national d'identité, photo, email, téléphone, etc.).
8. **Vérification KYC** : pour répondre aux exigences réglementaires (voir section 6), certains niveaux d'authentification imposent la collecte de pièces d'identité. Le back-office offre un module de validation manuelle des documents (scan de carte d'identité, permis de conduire, etc.). L'état KYC d'un utilisateur (ex. "en attente", "validé", "refusé") est stocké et affiché.
9. Gestion du profil : modification des coordonnées, changement de mot de passe, ajout/modification de méthodes de paiement (liaison compte bancaire ou mobile money).
10. Gestion des rôles : possibilité de comptes marchands distincts des comptes simples, avec un niveau de privilèges (par ex. plus grands volumes, frais réduits).

11. **Raison** : tout utilisateur doit pouvoir créer et gérer son profil de manière autonome, tout en permettant aux administrateurs d'effectuer le support et la validation (KYC) nécessaires à la conformité.

12. **Wallet numérique et solde** :

13. Chaque utilisateur possède un ou plusieurs portefeuilles électroniques (*wallets*) en différentes devises (au moins XOF et NGN). Chaque wallet conserve le **solde** et l'**historique des opérations** (crédit, débit).
14. Services de **consultation de solde** en temps réel. Toute transaction (retrait, dépôt, paiement) met à jour le solde avec cohérence.
15. Fonction de **cashing / rechargement** : via dépôt d'argent dans le wallet. Ceci peut se faire par :
    - *Dépôt Mobile Money* : l'utilisateur envoie de l'argent depuis son compte Mobile Money (Orange Money, MoMo…) vers un compte agrégé de la plateforme. L'API interroge le statut de la transaction et crédite le wallet en XOF ou NGN selon le type.
    - *Paiement par carte ou virement* (éventuellement en phase ultérieure ou via intégration, soumis à PCI DSS).
16. Fonction de **retrait** : inverse du dépôt, l'utilisateur peut transférer son solde du wallet vers son compte Mobile Money ou bancaire (via l'API de retrait du prestataire). Le débit est alors enregistré. Des frais peuvent être appliqués (voir ci-après).
17. Gestion de **plafonds** et limites journalières, imposés selon le niveau KYC et les régulations.

18. **Raison** : le wallet est le cœur du système monétaire. Il doit être fiable, toujours cohérent (transactions atomiques), et performant pour supporter de nombreuses opérations en simultané.

19. **Transfert d'argent P2P (nationaux au MVP)** :

20. Possibilité pour un utilisateur de transférer de l'argent (de son wallet) vers un autre utilisateur enregistré sur la plateforme. Le transfert se fait dans la même devise (XOF à XOF ou NGN à NGN).
21. Interface de sélection de bénéficiaire (par identifiant, téléphone, QR code généré ou scanné). Confirmation par OTP ou code PIN peut être exigée pour chaque transaction.
22. Mise à jour instantanée des soldes (débiteur/créditeur) avec enregistrement de la transaction.
23. Messages de notification (push/SMS/email) à l'émetteur et au bénéficiaire lors du transfert.
24. Restrictions éventuelles : limites de montants (quotas), vérifications anti-fraude.

25. **Raison** : en MVP, ce module permet l'usage fondamental de transfert d'argent entre utilisateurs locaux, favorisant les transactions en milieu personnel ou professionnel.

26. **Intégration Mobile Money (dépôt, retrait)** :

27. Connecteurs aux opérateurs de Mobile Money locaux (Orange Money, MTN Mobile Money, etc.) et potentiellement à des agrégateurs panafricains (CinetPay, PayDunya, etc.) pour les opérations financières.
28. **Dépôt depuis Mobile Money** : l'utilisateur initie le dépôt dans son application opérateur, la plateforme enregistre la requête (ID de transaction). L'API vérifie l'état via le callback du fournisseur et crédite le wallet une fois confirmé.
29. **Retrait vers Mobile Money** : l'utilisateur demande un retrait, la plateforme exécute une API de transfert vers son compte Mobile Money. Après confirmation, le wallet est débité.
30. Gestion des statuts asynchrones (succès, échec, en attente) avec notifications appropriées.
31. Respect des délais imposés par les opérateurs (quelques secondes à minutes).

32. **Raison** : le Mobile Money est extrêmement utilisé au Bénin. L'intégration directe permet aux utilisateurs de facilement approvisionner leur wallet et d'en retirer, en profitant des canaux existants. (Exemple : Orange Money comptait plus de 20 millions d'utilisateurs en Afrique de l'Ouest/Centre en 2021 [1].)

33. **Paiement marchand (QR code et lien de paiement)** :



Figure 2. Exemple de paiement marchand mobile sans contact (illustration).

34. **QR code marchand** : chaque marchand peut générer un code QR unique lié à son compte. Le client scanne ce code avec son application (web ou mobile) pour initier le paiement du montant indiqué. Le backend traite la transaction (vérifie solde, débite l'utilisateur, crédite le marchand) et notifie les deux parties.
35. **Lien de paiement** : pour les ventes en ligne, la plateforme permet la génération de liens de paiement contenant le montant et la référence. L'utilisateur final clique sur ce lien (SMS, email, site web du marchand) et complète le paiement dans l'app ou le navigateur.
36. En cas de paiement par carte bancaire (optionnel), un redirection vers une passerelle agréée conforme PCI DSS peut être prévue. Dans ce cas, la plateforme ne stocke aucune donnée sensible de carte, se reposant sur la tokenisation du prestataire [5] .
37. Tableau de bord marchand : consultation des transactions, solde marchands, et demande de virement du solde vers leur compte bancaire local (en fin de journée) via des batchs de paiement.

38. **Raison** : permet de rendre la plateforme attractive aux commerçants. Le paiement QR code favorise les transactions en physique sans terminaux coûteux, tandis que le paiement par lien facilite l'e-commerce.

39. **Gestion des frais de transaction (paramétrable par type d'utilisateur)** :

40. Système de tarification flexible : chaque type d'opération (transfert P2P, dépôt Mobile Money, retrait, paiement marchand) peut avoir une **commission** distincte. Les commissions peuvent dépendre du profil utilisateur (particulier, microentreprise, marchand), du montant, ou du volume d'activité (paliers de volume).
41. Interface back-office pour définir et ajuster dynamiquement ces frais (valeurs fixes ou pourcentages), avec date d'effet programmée. Par exemple : « 0,5% pour les transferts P2P de particuliers, 2% pour retraits vers Mobile Money, 0% pour dépôts Mobile Money au premier niveau KYC ».
42. Calcul et application automatiques des frais sur chaque transaction, avec journalisation détaillée (le reçu de transaction affiche le montant et les frais prélevés).

43. Support de modèles promotionnels (périodes sans frais) ou de frais négociés pour certains grands marchands.

44. **Raison** : pour la viabilité économique, il est crucial de pouvoir moduler les frais selon le marché et la stratégie. Ce module garantit que le tarif du service reste compétitif tout en couvrant les coûts opérationnels.

45. **Gestion de devises multiples (XOF, NGN)** :

46. Support natif d'au moins deux devises : le franc CFA (XOF, monnaie unique UEMOA, utilisée au Bénin) et le naira (NGN, Nigeria).
47. Chaque utilisateur peut disposer d'un wallet par devise. Les opérations intra-devises sont simples (voir modules ci-dessus). Pour les opérations en devises différentes, des règles claires seront définies (ex : interdiction ou simple référence de conversion manuelle par l'utilisateur). Dans un MVP, on privilégie l'absence de conversion automatique pour simplifier la conformité.
48. Affichage des montants et conversions éventuelles au taux du marché (via une API de taux de change) pour les interfaces.
49. Comptabilité multi-devises dans la base de données (entités « transactions », « soldes » liées à une devise).

50. **Raison** : bien qu'à l'initial le MVP couvre uniquement les transferts nationaux, la plateforme sera conçue pour accueillir des pays voisins (ex. extension future au Nigeria). La gestion multi-devises supporte cette évolutivité. Les opérateurs régionaux comme la BCEAO encouragent l'interopérabilité UEMOA et extérieures [6].

51. **Historique des transactions et export** :

52. Chaque utilisateur peut consulter en détail ses opérations passées (date, type, montant, contrepartie, frais). L'interface propose des filtres (date, type de transaction, montant) et une pagination efficace.
53. Téléchargement/export des relevés sous format CSV ou PDF (pour comptabilité ou service client). Les exports incluent un rappel des soldes précédents et suivants chaque transaction.
54. Pour les administrateurs, un historique global (avec droits d'accès) permet de tracer toute opération sur la plateforme, indispensable en cas de litige ou audit interne.

55. **Raison** : transparence et traçabilité sont essentielles en fintech. Les utilisateurs doivent suivre leur activité financière et exporter des relevés si besoin (par exemple pour un tiers de confiance ou KYC).

56. **Notifications (email, push, SMS)** :

    - Le système envoie des notifications pour divers événements : confirmation d'inscription, réception d'OTP, réussite/échec de transaction, rappel de solde faible, promotions ou alertes de sécurité.
    - **Email** : via un service SMTP ou API (SendGrid, Mailgun). Utilisé pour communication non urgente (reçus de transaction, newsletters).
    - **Push Mobile** : via FCM (Firebase Cloud Messaging) pour notifications en temps réel dans l'app mobile Flutter (transfert reçu, demande de paiement, etc.).
    - **SMS** : pour OTP et alertes critiques (confirmation de paiement, suspicion de fraude). L'intégration d'un fournisseur SMS local ou Twilio garantit la livraison rapide.
    - Les préférences de notification (type d'alertes, opt-in/opt-out) sont configurables dans le profil utilisateur.

- **Raison** : les notifications renforcent la sécurité (détection rapide de transactions non autorisées) et l'engagement utilisateur (rappels, promotions). Elles doivent être modulables et fiables sur tous les canaux (email/SMS/push).

57. **Système de « cercle de confiance » simplifié** :

    - Fonctionnalité sociale permettant à un groupe d'utilisateurs de définir des règles de transaction internes. Par exemple, un groupe familial ou professionnel où chaque transaction au-dessus d'un certain seuil nécessite l'approbation d'un ou plusieurs membres du groupe.
    - Un utilisateur peut créer un groupe (cercle) et inviter d'autres utilisateurs. On définit un *seuil de validation* et un *mode* (par ex. 2 personnes sur 3 doivent approuver une transaction avant exécution).
    - Lorsqu'un membre initie un transfert d'un montant conséquent, la transaction reste « en attente » et est affichée dans un fil d'attente du back-office de groupe. Les autres membres reçoivent une notification pour valider ou rejeter.
    - Une fois le quorum atteint, la transaction est exécutée automatiquement ou annulée. Les décisions sont enregistrées pour audit.
    - **Raison** : ce mécanisme, inspiré du concept de "trusted circle", augmente la confiance entre utilisateurs (ex. associations, coopératives, familles) et prévient les abus de gros montants sans contrôle.

58. **Dashboard administratif (back-office)** :

    - Interface web réservée aux administrateurs, accessible via un compte sécurisé (2FA). Peut être implémentée avec React Admin ou une application dédiée.
    - **Statistiques et reporting** : vue d'ensemble en temps réel du volume de transactions, nombre d'utilisateurs actifs, répartition des paiements par canal (P2P, Mobile Money, QR, etc.), indicateurs clés (KPIs) journaliers/mensuels. Graphiques interactifs (tableau de bord) pour le suivi de l'activité et la prise de décision.
    - **Gestion KYC et support client** : liste des utilisateurs en attente de validation, avec accès aux documents scannés. Boutons pour « Valider » ou « Rejeter » manuellement. Historique des actions KYC. Possibilité de communiquer avec l'utilisateur (email) pour demander info complémentaire.
    - **Paramétrage global** : gestion des frais (section 7 ci-dessus), des taux de change, des limites d'usage. Gestion des accès (créer/supprimer des comptes admin, assigner des rôles).
    - **Surveillance et audit** : logs détaillés (connexion des admins, modifications de configuration, anomalies). Mécanisme d'alertes (par exemple si un seuil de fraude est atteint).
    - **Gestion des incidents** : visibilité sur les transactions échouées ou en litige, avec outils pour relancer ou rembourser manuellement si nécessaire.
    - **Raison** : le back-office est crucial pour la gestion opérationnelle et la conformité de la plateforme. Il centralise le contrôle et permet aux responsables métiers/IT de piloter le service fintech (contrôle KYC, validation manuelle, paramétrage métier).

5. Ordre de développement conseillé (priorisation)

Pour livrer un MVP cohérent tout en validant rapidement les hypothèses métier, on propose l'ordre de développement suivant :

1. **Auth & Comptes Utilisateurs (Modules 1 & 2)** :
   Implémenter en premier lieu l'inscription, la connexion (avec OTP) et la gestion de profil. Sans ces bases, impossible de tester les autres fonctionnalités. On posera aussi les fondations de la sécurité (hash de mot de passe, tokens JWT, SSL).

2. **Wallet Numérique & Base de données (Module 3)** :
   Concevoir la structure de données des wallets et transactions. Développer la logique de solde (crédit/débit atomiques). Au moins un canal de rechargement simple (par exemple dépôt factice ou via Mobile Money en bac à sable).

3. **Transferts P2P (Module 4)** :
   Permettre les transactions entre utilisateurs enregistrés. Cela valide l'équilibre du système (2 cas simples : débit/crédit réciproques) et teste les notifications de base.

4. **Back-office de base (Module 12)** :
   Mettre en place une interface admin minimale pour voir les utilisateurs et transactions créés (sans KYC pour l'instant). Cela aide à superviser le travail déjà effectué et à ajouter des données de test.

5. **Intégration Mobile Money (Module 5)** :
   Intégrer un ou plusieurs services réels de Mobile Money pour les dépôts/retraits. Cela nécessite de travailler avec de vrais partenaires ou sandbox, donc commencez tôt (pouvant prendre du temps de certificat). Valider le workflow de bout en bout : dépôt SMS -> confirmation -> crédit sur wallet.

6. **Paiement marchand (Module 6)** :
   Une fois les transferts et dépôt OK, ajouter la génération de QR codes et liens de paiement. Tester la réception de fonds par les marchands. Ce module est critique pour l'adoption commerçante.

7. **Gestion des frais (Module 7) et Devises (Module 8)** :
   Implémenter le calcul dynamique des frais et la possibilité d'avoir plusieurs devises. Initialement, on peut fixer un schéma de frais basique (ex. taux fixe) pour tester le revenu. La gestion multi-devise peut être simplifiée (un seul wallet par devise sans conversion) dans un premier temps.

8. **Historique & Export (Module 9)** :
   Ajouter des pages pour consulter l'historique complet des transactions et exporter les données. Ce module est nécessaire pour la version produit (KYC, comptabilité).

9. **Notifications avancées (Module 10)** :
   Après les opérations critiques, mettre en place l'envoi d'emails, de SMS OTP (si pas déjà pour

login) et de push mobile. Développer un template d'email et l'infrastructure push pour l'app Flutter.

10. **Cercle de confiance (Module 11)** :
    Ce cas d'usage peut être ajouté vers la fin du MVP, car il est moins prioritaire que les fonctions de base. Lors de son implémentation, il doit utiliser les mécanismes de workflow transactionnel déjà en place.

11. **Sécurisation et tests complémentaires** :
    À chaque étape, écrire des tests (unitaires, d'intégration) et effectuer des audits de sécurité (tests de pénétration, revue de code).

Cette priorisation technique permet de déployer progressivement une solution fonctionnelle à chaque phase, d'obtenir rapidement un produit testable (pour ajuster aux retours) et de valider les points d'intégration sensibles (Mobile Money, QR).

6. Contraintes techniques et réglementaires

- **Régulations BCEAO et UEMOA** : la plateforme doit se conformer aux directives de la Banque Centrale des États de l'Afrique de l'Ouest. Notamment, les institutions émettrices de monnaie électronique doivent être agrées et respecter les nouvelles règles (loi bancaire révisée) [7]. Cela inclut un *e-KYC* renforcé pour l'identification à distance des clients, la surveillance de portefeuilles inactifs, et l'acheminement de toutes les transactions nationales via des plateformes autorisées par la BCEAO [6]. Par exemple, les transactions en XOF doivent transiter via les canaux UEMOA (SICA-UEMOA) si un règlement interbancaire est nécessaire.

- **Conformité PCI DSS** : si la plateforme gère des paiements par cartes bancaires (via QR code ou lien), elle doit respecter la norme PCI DSS. Concrètement, **aucune information de carte de crédit complète ne doit être stockée** sur nos serveurs ; utilisation de tokenisation et chiffrement de bout en bout est obligatoire. Selon HUB2, la conformité PCI implique des centres de données sécurisés, des contrôles d'accès stricts, le chiffrement et la tokenisation des données sensibles, ainsi que des audits réguliers de vulnérabilités [5]. On doit donc prévoir un partage de charge (Load Balancer), WAF, et un protocole TLS 1.2+ sur toutes les communications pour assurer la confidentialité.

- **KYC/AML** : conformément aux standards internationaux (GAFI) et africains, la plateforme intégrera des procédures KYC graduées (p.ex. seuils UEMOA : KYC simplifié pour faibles montants, KYC avancé au-delà d'un certain plafond). Les données personnelles collectées doivent être protégées (cryptage en base). Un registre des opérations suspectes doit être maintenu, et l'équipe opérationnelle doit être formée aux obligations de lutte anti-blanchiment.

- **Contraintes techniques** :

- *Compatibilité device* : l'application mobile Flutter doit fonctionner sur Android (version minimale à définir) et iOS, en adoptant un design responsive et ergonomique. Le Web Next.js doit être responsive et testé sur plusieurs navigateurs (Chrome, Firefox, Edge).
- *Scalabilité et disponibilité* : viser une architecture distribuée (microservices ou modulaires) afin de supporter la montée en charge. Par référence, des acteurs fintech globaux conçoivent leurs systèmes pour des milliers de transactions par seconde avec 99,999% de disponibilité [8]. Même

si notre MVP cible un trafic initial plus modeste, l'infrastructure doit rester évolutive (conteneurs Kubernetes, base de données clusterisée).
- *Interopérabilité* : usage d'APIs RESTful bien documentées (Swagger/OpenAPI) pour toutes les communications internes et externes. Utilisation de websockets ou MQTT si notifications temps réel en mode push sont implémentées.
- *Normes de développement* : code propre, modulable, tests unitaires/intégration obligatoires. Respect des principes OWASP Top 10 (prévention SQLi, XSS, CSRF…).
- *Sécurité des données* : chiffrement AES-256 des données sensibles au repos, TLS pour les données en transit. Stockage sécurisé des clés et certificats (HSM ou équivalent). Journalisation sécurisée des accès administratifs (impossible de masquer les logs).

En résumé, la plateforme doit répondre aux exigences BCEAO (e-KYC, clearing domestique), conserver la conformité PCI DSS pour toute intégration carte, et implémenter des mesures KYC/AML rigoureuses. Ces contraintes impliquent une forte implication de l'équipe projet sur la sécurité et la conformité dès les premières étapes.

7. Exigences de performance et de sécurité

- **Disponibilité et fiabilité** : le service doit être accessible 24/7 avec un objectif de service élevé (uptime $\geq$ 99,9 % en production). Le déploiement en cloud avec redondance multi-zone garantira la continuité de service. On envisage des déploiements sur un cluster Kubernetes, avec des probes de santé et un autoscaling pour absorber les pics (par ex. lors de promotions ou événements spéciaux). Selon l'envergure visée, des plateformes comme Nuvei planifient des architectures pour plus de *10 000 transactions par seconde* et 99,999 % de disponibilité [8]. Pour notre MVP, on dimensionnera l'infrastructure pour plusieurs centaines de TPS, avec possibilité de montée en charge rapide si besoin.

- **Temps de réponse** : les APIs critiques (authentification, consultation de solde, transfert d'argent) doivent répondre en <200 ms sous charge nominale pour garantir une bonne UX. Les opérations asynchrones (retraits Mobile Money, génération de QR) peuvent prendre quelques secondes à minutes selon l'API externe, mais le frontend devra afficher des indicateurs d'état (« en cours de traitement »).

- **Sécurité des transactions** :

- **Authentification et sessions** : utiliser des tokens JWT signés pour les sessions, avec expiration courte. Les OTP et mots de passe doivent être traités en unidirectionnel (hachage bcrypt).
- **Cryptographie** : chiffrer toute donnée sensible (PII, photos d'identité) en base et lors des transferts vers des services de stockage externes (S3). Utiliser HTTPS partout.
- **Défense multi-couches** : pare-feu applicatif (WAF), système de détection d'intrusion (IDS/IPS), audits de sécurité réguliers (tests d'intrusion, revues de code) et « bug bounty » en phase ouverte.
- **Gestion des vulnérabilités** : mises à jour automatiques des dépendances (Scan de vulnérabilités Node.js), policy de sécurité de contenu (CSP) pour les frontends.

- **Backup et reprise d'activité** : backups réguliers de la base de données (journalisés) et procédures de restauration en cas de sinistre.

- **Protection contre la fraude** :

- Analyse des transactions suspectes (montants élevés, fréquences inhabituelles) avec alertes automatisées.
- Verrouillage automatique du compte en cas de comportements anormaux (trois tentatives de paiement échouées, par exemple).
- Exigence de confirmer les paiements importants via double confirmation (deux OTP) pour des sommes au-delà d'un seuil.
- **Tests de charge et monitoring** : effectuer des tests de charge préliminaires (ex. 1000 utilisateurs simultanés) pour vérifier la tenue de la charge. Mettre en place du monitoring en temps réel (CPU, mémoire, latence des requêtes) et des alertes (utilisation CPU > 80%, erreurs 5xx, etc.). Les logs doivent être agrégés (ex. ELK) pour déboguer et tracer les transactions.

Ces exigences garantissent que la plateforme est robuste (haute disponibilité, tolérance aux pannes), rapide (bonnes performances sous charge) et **sécurisée au plus haut niveau** (cryptage, audits, conformité). Les indicateurs de performance et de sécurité seront documentés et testés en fin de sprint afin d'atteindre les SLA spécifiés.

8. Roadmap MVP et phasage clair

Pour structurer le développement et le lancement du MVP, voici un planning en *phases graduées* (chacune correspondant à des livrables intermédiaires) :

Phase	Fonctions clés déployées	Livrables intermédiaires	Durée indicative
Phase 0 (planification)	– Cahier des charges validé – Architecture détaillée et choix techniques (CI/CD, stack).	Document d'architecture et spécifications fonctionnelles détaillées.	2 semaines
Phase 1 (Fondations)	Authentification + Gestion comptes utilisateurs Wallet (solde, dépôts factices) API backend de base.	Version alpha : utilisateurs peuvent s'inscrire, se connecter, voir solde fictif, effectuer des transferts tests.	4 semaines
Phase 2 (Transferts)	Transferts P2P (débit/crédit réel), Historique de transactions Notifications basiques (email/SMS pour OTP).	Bêta interne : transferts entre comptes, vérification du bon calcul des soldes et de l'historique. Rapport de tests QA.	4 semaines
Phase 3 (Intégrations de paiement)	Intégration Mobile Money (dépôt et retrait réels) Paiement marchand QR code + liens (sans passerelle CB)	Prototype fonctionnel : dépôt/rétait via Orange Money (ou sandbox), création/scanning de QR, paiements marchands. Tests d'usabilité avec quelques commerçants pilotes.	4 semaines

Phase	Fonctions clés déployées	Livrables intermédiaires	Durée indicative
Phase 4 (Affinage et extensions)	Gestion des frais paramétrable Support multi-devise (XOF, implémentation NGN test) Notifications avancées (push mobile). Cercle de confiance (basique).	Version release candidate : structure de tarification en place, switching de devise possible, groupe de confiance implémenté et testé.	3 semaines
Phase 5 (Back-office & conformité)	Back-office complet (statistiques, KYC, gestion utilisateurs) Performance optimisée Tests de sécurité complets (pentest).	MVP en production (version 1.0) : plateforme déployée en ligne (production), accompagnée de la documentation utilisateur et technique. Rapport de conformité (PCI, KYC, etc.).	3 semaines

Chaque phase doit se terminer par un point de revue formel. Le MVP final doit inclure **les livrables code et documentation suivants** :
- Code source complet (Next.js, Flutter, NestJS, back-office) versionné (Git).
- API documentation (Swagger/OpenAPI) détaillant chaque endpoint.
- Manuels utilisateur et d'installation (config de l'application, déploiement).
- Maquettes ou prototypes UI/UX pour référence (wireframes ou designs finaux).
- Rapport de tests (unitaires, d'intégration, performance, sécurité) incluant les métriques atteintes.

Cette roadmap MVP séquencée permet de livrer rapidement une version utilisable (Phase 3) pour recueillir des retours utilisateurs (tests terrain, retours de commerçants pilotes), puis d'enrichir et durcir le système avant le lancement officiel (Phase 5).

9. Livrables attendus

À l'issue du projet, les éléments suivants doivent être fournis :

- **Code source complet**, bien documenté et versionné (ex. dépôt Git), comprenant :
- Frontend Web (Next.js) et mobile (Flutter) avec README d'utilisation.
- Backend (NestJS) avec scripts de déploiement et configuration (Dockerfiles, manifestes Kubernetes si utilisés).
- Application back-office (React Admin ou autre) avec son propre repo/documentation.
- **Documentation technique** :
- Spécifications techniques (diagrammes d'architecture, schéma de base de données, documentation des modules).
- Guide d'installation et de déploiement (configuration des environnements dev/staging/production).
- Guide d'utilisation API (exemple de requêtes, réponses, codes d'erreur).
- Documentation sur la sécurité et la conformité (traitement des données sensibles, procédures KYC).

- **Documentation fonctionnelle** :
- Manuel utilisateur final (explication des fonctionnalités principales pour les utilisateurs finaux et les marchands).
- Manuel d'administration (instructions pour les administrateurs sur l'utilisation du back-office, le traitement KYC, le paramétrage des frais, etc.).
- **Interfaces et maquettes** : prototypes ou captures d'écran de l'interface (fichiers design Figma/Sketch ou équivalent).
- **Rapports de tests** : résultats des tests unitaires, de performance (charge), et de sécurité (rapport de pentest).
- **Environnement de démonstration** : un environnement staging ou sandbox accessible (URL) pour démonstration des fonctionnalités clés.

Ces livrables permettront à la fois la validation du cahier des charges (respect des engagements fonctionnels et réglementaires) et la reprise du projet pour de futures évolutions.

Sources : analyses du contexte fintech en Afrique [1], normes BCEAO sur l'e-KYC et le paiement [9] [6], exigences PCI DSS pour le paiement sécurisé [5], et meilleures pratiques de sécurité OTP [2] et haute disponibilité système [8].

---

[1] Plateforme de paiement mobile en Afrique : Tendances, solutions et le rôle de HUB2 | Hub2
https://www.hub2.io/plateforme-de-paiement-mobile-en-afrique-tendances-solutions-et-le-role-de-hub2/

[2] [3] [4] One Time Password: How OTP Authentication Works | Okta
https://www.okta.com/blog/identity-security/what-is-a-one-time-password-otp/

[5] PCI DSS Compliance | Hub2
https://www.hub2.io/fr/pci-dss-compliance/

[6] [7] [9] bceao.int
https://www.bceao.int/sites/default/files/2024-12/BCEAO_2023_Annual_Report.pdf

[8] Nuvei Expands Partnership with Microsoft to Scale Global Payments Infrastructure to Support More Than 10,000 transactions per second | Nuvei
https://www.nuvei.com/posts/nuvei-expands-partnership-with-microsoft-to-scale-global-payments-infrastructure-to-support-more-than-10-000-transactions-per-second